

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : CSA0620 –Design and Analysis of Algorithms

A. Assignment Information

Particular	Details
Student Name:	U.UDHAYASHREE
Registration Number:	192521448
Department:	INFORMATION TECHNOLOGY
Programme:	B.E/B.Tech INFORMATION TECHNOLOGY
Course Code & Course Name	CSA0620 – Design and Analysis of Algorithms
Academic Year / Batch	2026-27
Faculty Name	Dr.A.Sairam
Assignment Title	Efficient Nearest-Neighbour Detection for Air-Traffic Collision Alerts (Divide-and-Conquer)
Date of Issue	02-09-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4, CO7
Bloom's Taxonomy Level	L4 – L5 (Analyzing, Evaluating)
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Air-traffic control systems and GPS-based fleet trackers must continuously check thousands of aircraft or vehicle positions for dangerously close proximity; this assignment mirrors that real-

	time collision-detection problem.
--	-----------------------------------

B. Assignment Problem / Challenge

In modern air-traffic monitoring systems, a large number of aircraft may be present within a monitored airspace at the same time. To maintain flight safety, it is important to identify aircraft that are closest to each other so that potential proximity or collision situations can be detected at an early stage. As the number of aircraft increases, checking the distance between every possible pair becomes computationally expensive.

The challenge is to design and implement an efficient **nearest-neighbour/closest-pair detection system** in which each aircraft is represented as a two-dimensional point. The system must determine the pair of aircraft having the minimum Euclidean distance and provide the distance between them.

Two algorithmic approaches are considered for solving the problem. The first approach uses a **Brute-Force method**, which compares every possible pair of aircraft. The second approach uses a **Divide-and-Conquer method**, which divides the set of aircraft into smaller regions, solves the subproblems recursively, and combines the results to identify the closest pair.

The assignment requires implementation and comparison of both approaches based on **execution time, number of distance comparisons, memory utilisation and scalability** for increasing numbers of aircraft. Uniformly distributed and clustered synthetic datasets are used to evaluate the algorithms under different spatial conditions.

The major engineering challenge is to determine which algorithm provides better performance while maintaining correctness. The results are analysed theoretically using asymptotic complexity and experimentally using measured performance data. The final solution is justified based on computational efficiency, scalability, reliability and suitability for an air-traffic monitoring application.

The proposed work demonstrates how an appropriate algorithmic design can reduce unnecessary computation and support faster processing of proximity information in safety-critical monitoring environments. The study also considers factors such as system reliability, computational resources, sustainability, safety, ethical responsibility and the need for human oversight in real-world deployment.

C. Problem Statement

Problem / Case / Design Challenge:

SkyGuard Air-Traffic Systems monitors n aircraft flying within a regional airspace, each represented as a point (x, y) on a 2-D radar plane. To prevent collisions, the system must continuously identify the two aircraft that are currently closest to each other and raise a proximity alert if their separation falls below a safety threshold. Because the number of aircraft can be large and the check must run in real time on every radar sweep, a naive pairwise comparison quickly becomes too slow. The system needs an efficient algorithm that finds the closest pair of points among n aircraft positions.

Your assignment has three linked parts:

- Part A – Brute-Force Baseline: Given n aircraft positions as (x, y) coordinates, implement a Brute-Force algorithm that compares every pair of points and returns the pair with the minimum Euclidean distance. Use the sample dataset in Annexure A to hand-compute all pairwise distances and identify the closest pair, establishing a correctness baseline for Part B.
- Part B – Divide-and-Conquer Closest Pair: Design a Divide-and-Conquer algorithm that sorts points by x -coordinate, recursively finds the closest pair in the left and right halves, and merges the results by checking only the points lying within a narrow vertical strip around the dividing line (each point compared against at most a small constant number of neighbours when the strip is sorted by y -coordinate). Trace the recursion on the Annexure A dataset and show that it produces the same closest pair found in Part A.
- Part C – Complexity and Trade-off Analysis: Derive the time and space complexity of both algorithms using Big-O notation, showing why Brute-Force is $O(n^2)$ while the Divide-and-Conquer approach achieves $O(n \log n)$. Discuss the constant-factor overheads of recursion and the strip-merge step, and identify the point sizes and real-time constraints under which each approach is the more practical choice for an operational air-traffic system.

Annexure A – Sample Dataset (aircraft positions, to be used for the hand-traced brute-force and divide-and-conquer walkthrough; students may also generate a larger randomized point set of at least $n = 15$ aircraft for the implementation and complexity/performance analysis in Sections 5–7):

Point	P1	P2	P3	P4	P5	P6	P7
X-coord	2	12	40	5	12	3	30
Y-coord	3	30	50	1	10	4	30

D. Requirements and Constraints

- **Functional Requirements**

- Represent each aircraft as a two-dimensional point (x, y).
- Accept the same set of aircraft coordinates for both algorithms.
- Determine the closest pair of aircraft and calculate their Euclidean distance.
- Generate an alert condition when the minimum distance is below a specified safety threshold.
- Record distance comparisons so the practical work can be compared with theoretical complexity.

- **Performance Requirements**

- Implement a brute-force $O(n^2)$ solution as the baseline.
- Implement a divide-and-conquer $O(n \log n)$ solution.
- Evaluate both methods for increasing input sizes.
- Measure execution time, distance comparisons, memory utilisation and scalability.
- Use identical input data for both methods to ensure a fair comparison.

- **Technical Requirements**

- Python is used for implementation and experimentation.
- The implementation uses standard Python modules such as math, random, time and tracemalloc.
- Synthetic aircraft coordinates are generated within a fixed 1000×1000 region.
- Both uniform and clustered point distributions are tested.

- **Cost, Resource and Safety Constraints**

- The solution should run on an ordinary computer without specialised hardware.
- The method should avoid unnecessary computation so that larger aircraft sets can be processed efficiently.
- Correctness is essential because a missed close pair could result in a missed collision warning.
- The algorithm is treated as a decision-support component; real deployment would

require additional certified safety mechanisms.

- **Sustainability, Environmental and Ethical Constraints**

- Reducing computational work can reduce processor time and energy consumption.
- The system should provide consistent and explainable results.
- Synthetic data is used for development and testing to avoid unnecessary exposure of sensitive operational data.
- Any real-world deployment must consider aviation safety standards, privacy, accountability and human oversight.

E. Student Work

1. Problem Understanding and Formulation

- The engineering problem is to identify the pair of aircraft that are closest to one another in a two-dimensional monitoring region. Each aircraft is represented by a point $P_i = (x_i, y_i)$. The required output is the pair (P_i, P_j) having minimum Euclidean distance.
- The Euclidean distance between two aircraft is $d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. If the minimum distance is less than a predefined safety threshold, the monitoring system can raise a proximity or collision alert.
- The main algorithmic challenge is that checking every pair requires $\Theta(n^2)$ distance calculations. For large numbers of aircraft, this becomes expensive. Therefore, a divide-and-conquer strategy is investigated to reduce the asymptotic running time to $O(n \log n)$.

2. Application of Course Knowledge

- The project applies the Design and Analysis of Algorithms concepts of brute-force search, divide-and-conquer, recurrence relations, asymptotic analysis, performance measurement and algorithm comparison.
- Brute force: compare every possible pair of points.
- Divide and conquer: sort points by x-coordinate, divide them into left and right halves, recursively solve each half, and examine only the relevant strip around the division line.
- Recurrence: $T(n) = 2T(n/2) + O(n)$, giving $O(n \log n)$ when the points are maintained in appropriate sorted order.

- Asymptotic comparison: $O(n^2)$ versus $O(n \log n)$.
- Experimental validation: measure execution time, number of comparisons and peak memory usage for increasing n .

3. Solution / Design / Methodology

Two solutions are developed. The first is a brute-force baseline. The second uses divide and conquer to reduce the number of distance comparisons.

- **Solution 1 – Brute Force**

For every aircraft i , the algorithm compares it with every aircraft $j > i$. The smallest distance found is retained. This method is simple and guarantees correctness, but the number of comparisons is $n(n-1)/2$.

- **Solution 2 – Divide and Conquer**

The aircraft points are sorted by x -coordinate and divided into two approximately equal halves. The closest pair in each half is found recursively. Let δ be the smaller of the two distances. A vertical strip of width 2δ around the dividing line is then examined. Only points in this strip need to be checked for a cross-boundary closest pair.

- **Dataset Methodology**

Synthetic points are generated in a 1000×1000 region. Two distributions are used: uniform points and clustered points. The same generated input is supplied to both algorithms for each experiment. Input sizes of 50, 100, 250, 500, 1000 and 2000 points are used.

Brute-Force Pseudocode

➤ BRUTE-FORCE-CLOSEST-PAIR(P)

```

minDistance  $\leftarrow \infty$ 
bestPair  $\leftarrow$  null
for  $i \leftarrow 0$  to  $n-2$ 
    for  $j \leftarrow i+1$  to  $n-1$ 
         $d \leftarrow \text{distance}(P[i], P[j])$ 
        if  $d < \text{minDistance}$ 
            minDistance  $\leftarrow d$ 
            bestPair  $\leftarrow (P[i], P[j])$ 
return bestPair, minDistance

```

Divide-and-Conquer Pseudocode

- **DIVIDE-AND-CONQUER-CLOSEST-PAIR(P_x, P_y)**
 - if number of points ≤ 3
 - solve directly by brute force
 - divide P_x into left and right halves
 - divide P_y into corresponding left and right sets
 - (pairL, dL) $\leftarrow$ recursively solve left half
 - (pairR, dR) $\leftarrow$ recursively solve right half
 - $\delta \leftarrow \min(dL, dR)$
 - construct strip containing points within δ of division line
 - inspect the required neighbouring strip points
 - return the closest pair among left, right and strip candidates

Implementation / Source Code and Environment / Tools Used

- The implementation contains both algorithms in the same Python program. The experimental framework generates uniform and clustered datasets, runs both methods on identical inputs, records execution time, distance-comparison counts and peak memory usage, verifies that both algorithms return the same minimum distance, and produces comparison graphs.
- The divide-and-conquer implementation assumes unique coordinate pairs in the generated experimental dataset. A production implementation should explicitly handle duplicate points because duplicate coordinates produce a minimum distance of zero.

Test Cases and Expected / Actual Results

- Annexure A provides seven sample points: $P_1=(2,3)$, $P_2=(12,30)$, $P_3=(40,50)$, $P_4=(5,1)$, $P_5=(12,10)$, $P_6=(3,4)$, and $P_7=(30,30)$. The closest pair is P_1 and P_6 with distance $\sqrt{2} \approx 1.414$. The brute-force method performs 21 pair comparisons for $n=7$.

Experimental Test Cases

- Uniform distribution: points generated randomly throughout the fixed 1000×1000 region.
- Clustered distribution: points generated around four predefined centres and clipped to the same region.
- Input sizes: $n = 50, 100, 250, 500, 1000$ and 2000 .
- Validation criterion: both algorithms must return the same minimum distance for every test.

- Timing criterion: the minimum of three timing runs is recorded to reduce transient variation.

4. Use of Modern Tools

- Brute-force closest pair: easiest to implement and useful as a correctness baseline, but has quadratic growth.
- Divide-and-conquer closest pair: better asymptotic performance and suitable for scalability.
- A grid/spatial-partition approach could also reduce comparisons in practical systems, but its performance depends on suitable spatial assumptions and implementation choices.
- For this assignment, divide and conquer is selected as the preferred algorithm because it provides predictable $O(n \log n)$ scalability while remaining directly connected to the course concepts.
- Python – implementation and experimental evaluation.
- math – Euclidean distance calculations.
- random – synthetic dataset generation.
- time – execution-time measurement.
- tracemalloc – peak memory measurement.
- matplotlib – performance graphs.
- Git/GitHub – source-code versioning and project submission.

5. Results and Validation

Implementation


```

import math
import random
import time
import tracemalloc

# =====
# DISTANCE CALCULATION
# =====

def distance(p1, p2):
    return math.sqrt(
        (p1[0] - p2[0]) ** 2 +
        (p1[1] - p2[1]) ** 2
    )

# =====
# BRUTE-FORCE METHOD
# Time Complexity: O(n^2)
# =====

def brute_force(points):
    n = len(points)

    if n < 2:
        return None, float("inf"), 0

    min_distance = float("inf")
    closest_pair = None
    comparisons = 0

    for i in range(n - 1):
        for j in range(i + 1, n):
            comparisons += 1

            d = distance(points[i], points[j])

            if d < min_distance:
                min_distance = d
                closest_pair = (points[i], points[j])

    return closest_pair, min_distance, comparisons

# =====

```

```

def test_dataset(points, dataset_name):
    print("\n" + "=" * 75)
    print(dataset_name)
    print("=" * 75)

    print("Number of Points:", len(points))

    # Brute Force
    brute_result = measure_algorithm(brute_force, points)

    # Divide and Conquer
    dc_result = measure_algorithm(divide_and_conquer, points)

    brute_pair = brute_result[0]
    brute_distance = brute_result[1]
    brute_comparisons = brute_result[2]
    brute_time = brute_result[3]
    brute_memory = brute_result[4]

    dc_pair = dc_result[0]
    dc_distance = dc_result[1]
    dc_comparisons = dc_result[2]
    dc_time = dc_result[3]
    dc_memory = dc_result[4]

    print("\nBRUTE-FORCE")
    print("-" * 40)
    print("Closest Pair      :", brute_pair)
    print("Minimum Distance   :", round(brute_distance, 6))
    print("Comparisons        :", brute_comparisons)
    print("Execution Time     :", round(brute_time, 4), "ms")
    print("Peak Memory        :", round(brute_memory, 2), "KB")

    print("\nDIVIDE-AND-CONQUER")
    print("-" * 40)
    print("Closest Pair      :", dc_pair)
    print("Minimum Distance   :", round(dc_distance, 6))
    print("Comparisons        :", dc_comparisons)
    print("Execution Time     :", round(dc_time, 4), "ms")
    print("Peak Memory        :", round(dc_memory, 2), "KB")

    # Validation
    if math.isclose(
        brute_distance,
        dc_distance,
        rel_tol=1e-9,
    ):

```

```
daa assignment.py - C:/Users/parva/daa assignment.py (3.11.9)
File Edit Format Run Options Window Help

# FINAL SUMMARY
# -----

print("\n\n")
print("-" * 95)
print("FINAL PERFORMANCE SUMMARY")
print("-" * 95)

print(
    f"{'N':>6}"
    f"{'BF Time (ms)':>15}"
    f"{'DC Time (ms)':>15}"
    f"{'BF Comp.':>15}"
    f"{'DC Comp.':>15}"
)

print("-" * 95)

for row in uniform_results:
    n = row[0]
    bf_time = row[1]
    dc_time = row[2]
    bf_comp = row[3]
    dc_comp = row[4]

    print(
        f"{'n':>6}"
        f"{'bf_time':>15.3f}"
        f"{'dc_time':>15.3f}"
        f"{'bf_comp':>15}"
        f"{'dc_comp':>15}"
    )

print("\n")
print("-" * 75)
print("PROGRAM COMPLETED SUCCESSFULLY")
print("-" * 75)

# =====
# RUN PROGRAM
# =====

if __name__ == "__main__":
    main()

IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/Users/parva/daa assignment.py =====
EFFICIENT NEAREST-NEIGHBOUR DETECTION
AIR-TRAFFIC COLLISION ALERT SYSTEM
=====

SAMPLE TEST CASE
=====
Closest Pair: ((2, 3), (3, 4))
Minimum Distance: 1.414214
Comparisons: 21

INPUT SIZE: 50
=====
UNIFORM DATASET
=====
Number of Points: 50

BRUTE-FORCE
=====
Closest Pair : ((997.5376064951103, 509.5262936764645), (996.1213802400968, 529.114345099137))
Minimum Distance : 19.639182
Comparisons : 1225
Execution Time : 2.7945 ms
Peak Memory : 0.26 KB

DIVIDE-AND-CONQUER
=====
Closest Pair : ((996.1213802400968, 529.114345099137), (997.5376064951103, 509.5262936764645))
Minimum Distance : 19.639182
Comparisons : 63
Execution Time : 1.1123 ms
Peak Memory : 11.77 KB

VALIDATION: PASSED
Both algorithms produced the same minimum distance.

=====
CLUSTERED DATASET
=====
Number of Points: 50
```

```
IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help

Closest Pair : ((413.04402219171976, 742.3666616628996), (413.95306533418574, 742.5846461655369))
Minimum Distance : 0.934814
Comparisons : 499500
Execution Time : 840.099 ms
Peak Memory : 0.86 KB

DIVIDE-AND-CONQUER
-----
Closest Pair : ((413.04402219171976, 742.3666616628996), (413.95306533418574, 742.5846461655369))
Minimum Distance : 0.934814
Comparisons : 1150
Execution Time : 18.2447 ms
Peak Memory : 161.42 KB

VALIDATION: PASSED
Both algorithms produced the same minimum distance.

=====
CLUSTERED DATASET
=====
Number of Points: 1000

BRUTE-FORCE
-----
Closest Pair : ((393.25014455941425, 763.6324240081478), (393.0156762931251, 763.7855431783123))
Minimum Distance : 0.280037
Comparisons : 499500
Execution Time : 840.8565 ms
Peak Memory : 0.86 KB

DIVIDE-AND-CONQUER
-----
Closest Pair : ((393.25014455941425, 763.6324240081478), (393.0156762931251, 763.7855431783123))
Minimum Distance : 0.280037
Comparisons : 1093
Execution Time : 18.5732 ms
Peak Memory : 161.2 KB

VALIDATION: PASSED
Both algorithms produced the same minimum distance.

INPUT SIZE: 2000

=====
UNIFORM DATASET
=====
```

Ln:20 Col:15

```
IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help

VALIDATION: PASSED
Both algorithms produced the same minimum distance.

=====
CLUSTERED DATASET
=====
Number of Points: 2000

BRUTE-FORCE
-----
Closest Pair : ((185.27255918762438, 210.86055586629888), (185.24225988476803, 210.8750911928322))
Minimum Distance : 0.033605
Comparisons : 1999000
Execution Time : 3590.3679 ms
Peak Memory : 0.86 KB

DIVIDE-AND-CONQUER
-----
Closest Pair : ((185.27255918762438, 210.86055586629888), (185.24225988476803, 210.8750911928322))
Minimum Distance : 0.033605
Comparisons : 2232
Execution Time : 41.4895 ms
Peak Memory : 300.54 KB

VALIDATION: PASSED
Both algorithms produced the same minimum distance.

=====
FINAL PERFORMANCE SUMMARY
=====
N      BF Time(ms)    DC Time(ms)    BF Comp.    DC Comp.
-----
50      2.795           1.112          1225         63
100     5.182           1.815          4950         128
250    30.601          4.413         31125         273
500   199.302         8.551        124750         559
1000  840.099        18.245       499500        1150
2000 3478.043       44.740     1999000       2320

=====
PROGRAM COMPLETED SUCCESSFULLY
=====
>>>
```

Ln:20 Col:15

Uniform Distribution Results

n	Brute Time (ms)	D&C Time (ms)	Brute Comparisons	D&C Comparisons	Brute Peak (KB)	D&C Peak (KB)
50	0.808	0.381	1225	63	0.3	7.62
100	3.292	0.698	4950	126	0.16	14.63

250	24.541	2.341	31125	253	0.16	35.18
500	143.22	5.01	124750	572	0.31	60.92
1000	583.412	11.066	499500	1150	0.34	130.64
2000	2453.014	23.952	1999000	2255	1.14	238.01

Clustered Distribution Results

n	Brute Time (ms)	D&C Time (ms)	Brute Comparisons	D&C Comparisons	Brute Peak (KB)	D&C Peak (KB)
50	0.78	0.339	1225	61	0.16	6.93
100	3.65	0.8	4950	124	0.16	11.52
250	23.62	2.356	31125	270	0.16	26.59
500	130.756	4.811	124750	560	0.31	46.91
1000	597.477	9.703	499500	1111	0.34	102.96
2000	2508.771	24.37	1999000	2231	1.19	238.45

Validation result: all tested input sets produced matching closest distances from the brute-force and divide-and-conquer implementations. The comparison counts demonstrate the expected quadratic growth of brute force and substantially lower comparison growth for divide and conquer.

6. Analysis and Engineering Decision

The brute-force algorithm is straightforward and has low implementation complexity. However, the number of pairwise comparisons grows as $n(n-1)/2$. At $n=2000$, this is 1,999,000 comparisons. The measured execution time consequently increases sharply.

The divide-and-conquer algorithm requires more careful implementation because the points must be partitioned and the strip handled correctly. Its comparison count and execution time grow much more slowly. In the experiments, the divide-and-conquer method is substantially faster at larger input sizes.

The trade-off is therefore simplicity versus scalability. Brute force is suitable for very small datasets and as a correctness reference. Divide and conquer is the better engineering decision for an air-traffic monitoring workload where the number of tracked aircraft may increase.

Engineering Decision

Select the divide-and-conquer algorithm for the scalable solution, while retaining the brute-force implementation as a validation baseline for testing and verification.

7. Broader Considerations

The work supports SDG 9 – Industry, Innovation and Infrastructure by applying an efficient algorithmic method to an infrastructure-oriented monitoring problem. More efficient computation can support timely processing of larger datasets and reduce unnecessary computational effort.

- Safety: closest-pair detection can contribute to early proximity warning, but it must be integrated with certified aviation systems.
- Environment: lower computation can reduce processor utilisation and associated energy demand.
- Society: timely detection can contribute to safer air-traffic operations.
- Ethics: operational data should be protected and system decisions should be auditable.
- Accessibility: results should be presented clearly to technical and operational users.
- Professional responsibility: testing, validation, documentation and human oversight are essential.

8. Conclusion

The project demonstrates that divide and conquer provides a major scalability advantage over brute-force closest-pair detection. Both algorithms correctly identify the closest pair on the tested datasets, while the divide-and-conquer method performs far fewer comparisons and has much lower execution time as the number of points increases.

Limitations

- The experiment uses synthetic two-dimensional coordinates rather than live aircraft trajectories.
- Execution time and memory values depend on the computer and Python runtime.
- The implementation does not model aircraft altitude, velocity, acceleration or time-to-collision.
- The basic implementation assumes unique point coordinates.

Possible Improvements

- Extend the model to three-dimensional coordinates including altitude.
- Incorporate aircraft velocity and trajectory prediction.
- Use streaming or spatial-index methods for continuously changing aircraft positions.

- Handle duplicate points explicitly.
- Evaluate larger and more realistic datasets.
- Integrate threshold-based alerting and visual monitoring.

9. Student Reflection

This assignment helped me understand how the concepts of Design and Analysis of Algorithms can be applied to a realistic engineering problem. The air-traffic collision alert scenario helped me understand the importance of efficiently identifying the closest pair of aircraft when the number of aircraft increases. Instead of only studying algorithmic complexity theoretically, I was able to apply the concepts to a practical computing problem.

I implemented and studied two approaches: the Brute-Force algorithm and the Divide-and-Conquer algorithm. The Brute-Force method was simple to understand and implement, but the number of distance comparisons increased rapidly with the input size. The Divide-and-Conquer approach required more careful design and recursive thinking, but it significantly reduced the number of comparisons and improved execution time for larger datasets.

The experimental analysis helped me understand the practical difference between $O(n^2)$ and $O(n \log n)$ complexity. Testing the algorithms using both uniform and clustered datasets also showed me the importance of evaluating an algorithm under different input conditions. Comparing execution time, distance comparisons and memory utilisation helped me make a justified engineering decision rather than selecting an algorithm only based on theory.

Through this assignment, I also improved my skills in Python programming, recursive algorithm implementation, performance measurement and result analysis. I learned that correctness, efficiency, scalability and reliability are all important when designing an algorithm for a safety-related application. The project also made me aware that real-world systems require additional considerations such as safety, ethical responsibility, data protection, sustainability and human supervision.

Overall, this assignment strengthened my understanding of divide-and-conquer techniques and improved my ability to analyse, implement, test and compare algorithms. It also helped me develop the confidence to apply algorithmic concepts to practical engineering problems and justify a solution using both theoretical and experimental evidence.

10. References

- Python Software Foundation. (2026). *Python 3.12.13 Documentation*. Python Documentation. [Python 3.12 Documentation](#)
- Python Software Foundation. (2026). *Python Documentation – Version History*. Python.org. [Python Release Documentation](#)
- Matplotlib Development Team. (2026). *Matplotlib Documentation – Matplotlib 3.11.1*. Matplotlib. [Matplotlib Documentation](#)
- Kramer, P. (2024). *Computational Geometry: Closest Pair – Divide and Conquer*. Technical University of Braunschweig.
- Katz, S. M., Moss, R. J., Asmar, D. M., Olson, W. A., Kuchar, J. K., & Kochenderfer, M. J. (2025). *Aircraft Collision Avoidance Systems: Technological Challenges and Solutions on the Path to Regulatory Acceptance*. arXiv.
- GeeksforGeeks. (2025). *Closest Pair of Points in C++*. Updated July 23, 2025.
- Python Software Foundation. (2026). *tracemalloc — Trace Memory Allocations*. Python 3.12 Documentation.

F. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.

Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates a limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.

		scalability with clear evidence.			
Performance Improvement, Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO1	PO1, PO2	L4	10

Algorithmic Solution Design	CO2	PO1, PO2, PO3	L4/L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO3, CO4	PO2, PO3, PO5	L5/L6	20
Implementation & Modern Tool Usage	CO4	PO3, PO5	L3/L6	15
Efficiency, Testing & Validation	CO2, CO6	PO2, PO4	L4/L5	15
Performance Improvement, Comparison & Final Decision	CO6, CO7	PO2, PO12	L4/L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO7	PO8, PO12	L4/L5	10

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous

Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

